

# Resource Allocation System – Iteration 35

## Participant Availability Rule for Committee Meeting Requests

Najib Attar

May 2026

### 1 Iteration Goal

The goal of Iteration 35 is to add participant availability checking for `CommitteeMeetingRequest`. In Iteration 34, committee meeting requests could store required participant IDs and roles, but the system did not yet verify whether those participants were available at the requested meeting time.

This iteration adds a new rule called `ParticipantAvailabilityRule`. The rule checks whether all required participants of a committee meeting are available during the preferred meeting time. This directly supports the professor's resource-sharing scenario where a student schedules a presentation or committee meeting that requires several professors to be available at the same time.

This iteration performs exact-time availability checking only. It does not implement automatic time suggestion, least-change optimization, global scheduling optimization, or automatic rescheduling.

### 2 Changes from Previous Iteration

The main change from Iteration 34 is the addition of participant availability validation inside the rule pipeline. The system now checks human-resource availability separately from physical room availability.

The following changes were introduced:

- Added `ParticipantAvailabilityRule`.
- Registered `ParticipantAvailabilityRule` in the rule pipeline.
- Updated `RuleEngineFacade` so participant availability is checked during request evaluation.
- Updated the `AllocationService` and main program flow so loaded users and user busy slots reach the rule engine.
- Reused `UserAvailabilityService` for working-hours and busy-slot availability logic.
- Preserved `AvailabilityRule` as the physical room availability rule.

- Added web dashboard support for creating `CommitteeMeetingRequest` rows and participant mappings.
- Preserved all existing allocation strategies without changing their algorithms.

### 3 Scenario

A student wants to schedule a master thesis presentation. The meeting requires a supervisor and committee members. The system must check two kinds of resources:

- The requested room must be available.
- All required human participants must be available.

For example, if Ali is busy on Monday from 10:00 to 12:00 and a committee meeting is requested for Monday from 10:00 to 11:00, the request must be rejected because a required participant is unavailable.

### 4 Modules and Their Tasks

| Module                                   | Task                                                                                                      |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>CommitteeMeetingRequest</code>     | Stores the meeting topic, preferred time slot, duration, required participant IDs, and participant roles. |
| <code>RequestParticipant</code>          | Represents the mapping between a committee meeting request and a required participant.                    |
| <code>UserBusySlot</code>                | Stores periods when a user is busy.                                                                       |
| <code>UserAvailabilityService</code>     | Determines whether a user is available based on working hours and busy-slot overlap.                      |
| <code>ParticipantAvailabilityRule</code> | Checks whether all required committee meeting participants are available.                                 |
| <code>AvailabilityRule</code>            | Checks physical room availability only.                                                                   |
| <code>RuleEngineFacade</code>            | Coordinates the rule pipeline and includes participant availability checking.                             |
| <code>AllocationService</code>           | Passes requests through the selected strategy and rule evaluation pipeline.                               |
| <code>DataController</code>              | Loads users, requests, request participants, and user busy slots.                                         |
| <code>Flask Dashboard</code>             | Allows committee meeting requests and participant mappings to be added from the web interface.            |

### 5 Participant Availability Rule Design

`ParticipantAvailabilityRule` is responsible for checking required human resources. It evaluates only `CommitteeMeetingRequest`. For all other request types, it passes immediately so that existing request behavior remains unchanged.

For a committee meeting request, the rule performs the following steps:

1. Read the preferred meeting time from `CommitteeMeetingRequest`.
2. Read the required participant IDs from the request.
3. For each required participant, find the matching user.
4. Use `UserAvailabilityService` to check whether the participant is available.
5. Reject the request if any participant is unavailable.
6. Return a clear rejection reason.

An example rejection reason is:

Required participant Ali is not available at Monday 10:00-11:00

This design keeps responsibility separated. `CommitteeMeetingRequest` stores meeting data, but it does not check availability. `UserAvailabilityService` performs availability computation, while `ParticipantAvailabilityRule` applies that computation to committee meeting validation.

## 6 Rule Pipeline Update

The participant availability rule was added after request-type and user-role checks, but before room-specific constraints. This order ensures that the system first checks whether the requester is allowed to make the request and use the requested space type before checking required human participants.

| Order | Rule                                     | Responsibility                                                                      |
|-------|------------------------------------------|-------------------------------------------------------------------------------------|
| 1     | <code>RequestTypeRule</code>             | Checks whether the requester can submit the request type.                           |
| 2     | <code>UserRoleRule</code>                | Checks whether the requester can use the requested space type.                      |
| 3     | <code>ParticipantAvailabilityRule</code> | Checks required participant availability for <code>CommitteeMeetingRequest</code> . |
| 4     | <code>CapacityRule</code>                | Checks room capacity.                                                               |
| 5     | <code>FeatureRule</code>                 | Checks required room features.                                                      |
| 6     | <code>StatusRule</code>                  | Checks room status.                                                                 |
| 7     | <code>LocationRule</code>                | Checks building/location requirement.                                               |
| 8     | <code>AvailabilityRule</code>            | Checks physical room time availability.                                             |

This separation is important because participant availability and room availability are different concerns. `ParticipantAvailabilityRule` checks people, while `AvailabilityRule` checks spaces.

## 7 Availability Policy

`UserAvailabilityService` applies the availability policy. Users are considered available only during working days and working hours, and only if the requested slot does not overlap a busy slot.

| Condition                                                        | Result      |
|------------------------------------------------------------------|-------------|
| Participant has no busy overlap and slot is within working hours | Available   |
| Participant has an overlapping busy slot                         | Unavailable |
| Requested slot is outside 09:00-17:00                            | Unavailable |
| Requested slot is on Saturday or Sunday                          | Unavailable |
| Requested slot is adjacent to a busy slot but does not overlap   | Available   |

The system uses half-open interval logic, written conceptually as  $[start, end)$ . This means that 09:00-10:00 and 10:00-11:00 do not conflict, but 10:00-11:00 and 10:30-11:30 do conflict.

## 8 Prototype Test Cases

The prototype was tested with committee meeting requests and existing request types.

| Request        | Type                                    | Condition                                    | Expected Result |
|----------------|-----------------------------------------|----------------------------------------------|-----------------|
| 40             | CommitteeMeeting                        | All required participants free               | Approved        |
| 41             | CommitteeMeeting                        | Required participant busy Monday 10:00-12:00 | Rejected        |
| Other requests | re- OneTime/Recurring/Existing behavior | Existing behavior                            | Unchanged       |

Request 40 represents a valid committee meeting. Its required participants are available, so the request is approved and allocated to M301 on Friday 13:00-14:00.

Request 41 represents a conflicting committee meeting. It includes a participant who is busy on Monday 10:00-12:00, while the requested meeting time is Monday 10:00-11:00. The request is rejected because the required participant is not available.



## 9 Iteration Comparison

| Feature                        | Before Iteration 35                                       | After Iteration 35                                  |
|--------------------------------|-----------------------------------------------------------|-----------------------------------------------------|
| Committee meeting participants | Stored as participant IDs and roles                       | Checked for availability during rule evaluation     |
| Human-resource availability    | Loaded but not used in committee meeting validation       | Enforced through <b>ParticipantAvailabilityRule</b> |
| Room availability              | Checked by <b>AvailabilityRule</b>                        | Still checked by <b>AvailabilityRule</b> only       |
| Allocation strategies          | Processed requests without participant availability logic | Remain unchanged and continue using the rule engine |
| Busy slot handling             | Available through <b>UserAvailabilityService</b>          | Reused by the participant rule                      |
| Time suggestion                | Not implemented                                           | Still not implemented                               |
| Least-change optimization      | Not implemented                                           | Still not implemented                               |

## 10 Updated Class Diagram

The updated class diagram includes `CommitteeMeetingRequest`, `RequestParticipant`, `ParticipantAvailabilityRule`, `UserAvailabilityService`, `UserBusySlot`, `TimeSlot`, `RuleEngine`, `RuleEngineFacade`, and `AvailabilityRule`. It shows that `CommitteeMeetingRequest` inherits from `Request`, stores required participant IDs, and is checked by `ParticipantAvailabilityRule`. It also shows that `ParticipantAvailabilityRule` uses `UserAvailabilityService`, which uses `UserBusySlot` data.

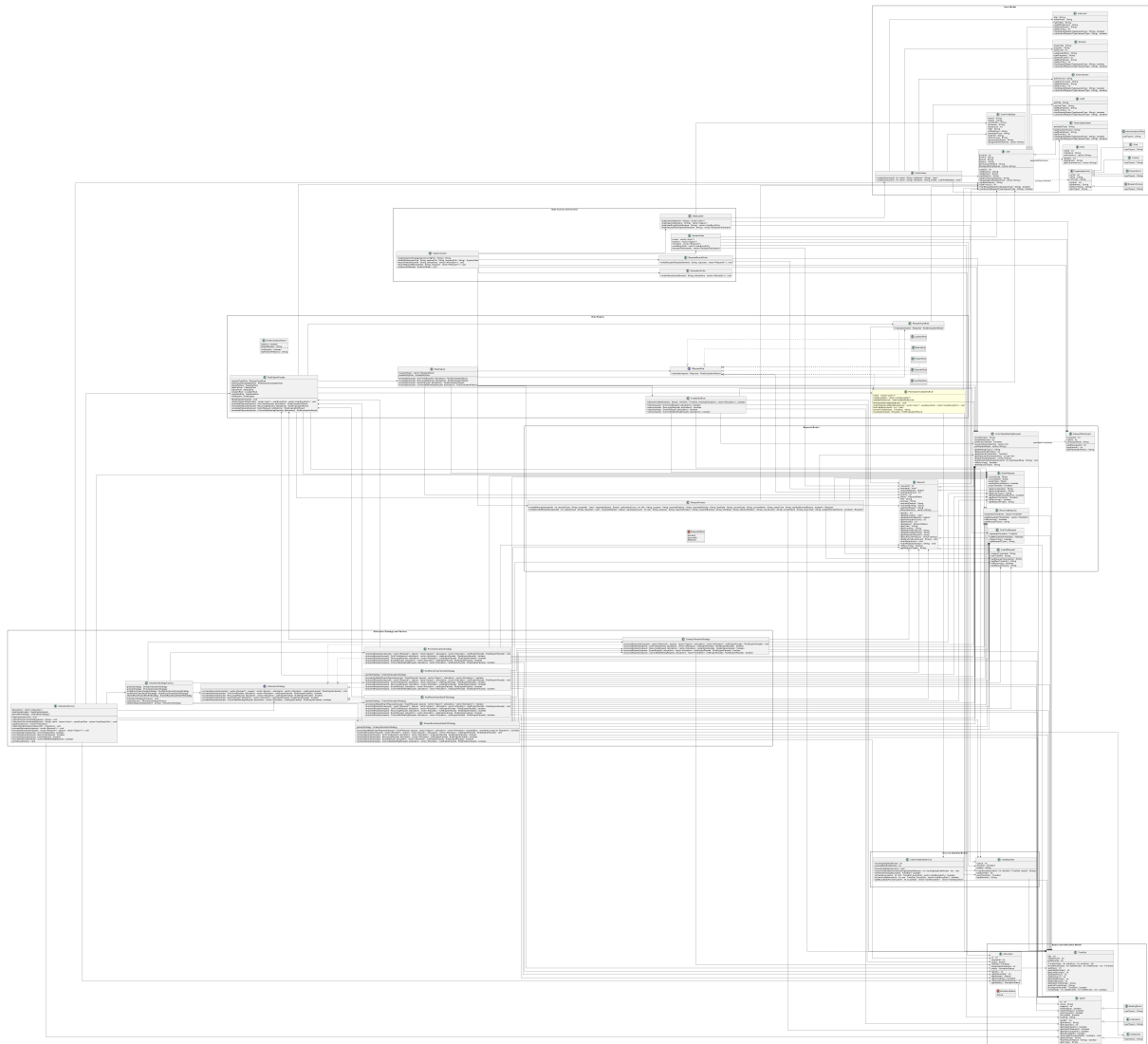


Figure 1: Updated class diagram for Iteration 35

## 11 Updated Workflow Diagram

The updated workflow shows that `DataController` loads requests, request participants, and user busy slots. `RequestFactory` creates `CommitteeMeetingRequest`, and `DataController` attaches participant mappings. During rule evaluation, `RuleEngineFacade` evaluates

RequestTypeRule, UserRoleRule, and then ParticipantAvailabilityRule. If any participant is unavailable, the request is rejected. If all participants are available, normal room rules continue.

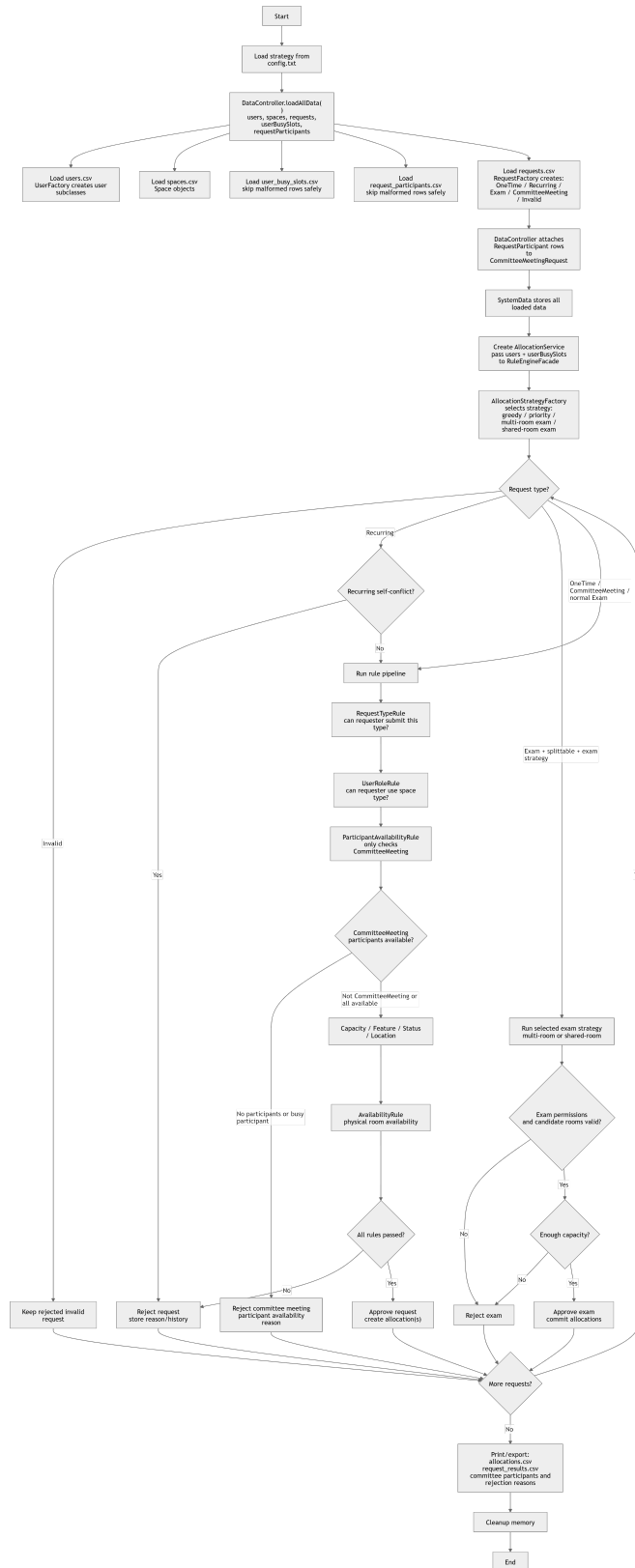


Figure 2: Updated workflow diagram for Iteration 35

## 12 Prototype Implementation

The prototype implementation confirms that participant availability is now part of the backend rule system. The rule uses the existing `UserAvailabilityService`, so working-hours logic and busy-slot overlap logic are not duplicated inside the rule.

The implementation also preserves the existing strategy pattern. Strategies still decide processing order and allocation behavior, but they do not directly check participant busy slots. This keeps validation inside the rule engine and allocation behavior inside strategies.

The Flask dashboard was also extended as a supporting interface. The Add Request page can create `CommitteeMeeting` rows, select required participants, and write participant mappings to `request_participants.csv`. The Allocation Summary page displays committee participants and backend rejection reasons. However, Flask does not perform participant availability checking; the C++ backend remains responsible for validation.

## 13 Summary

Iteration 35 adds exact-time participant availability checking for committee meeting requests. The new `ParticipantAvailabilityRule` checks whether all required participants are available at the requested time, using `UserAvailabilityService` and the loaded `user_busy_slots.csv` data.

The iteration keeps the design clean by separating responsibilities. `ParticipantAvailabilityRule` checks people, `AvailabilityRule` checks rooms, and allocation strategies remain unchanged. This prepares the system for future features such as common available time suggestions and least-change optimization without requiring a redesign of `CommitteeMeetingRequest` or the allocation strategies.